

CSCI1409 DEEP LEARNING

WEEK 2

Perceptron + Derivatives, Gradients & Backpropagation

Aidyn Abirov

KBTU, School of IT and Engineering

What the homework prepared

HOMEWORK	MAIN IDEA
Tasks 1–2	Local change and gradients
Task 3	The chain rule through a neuron
Tasks 4–5	Predictions, shapes and loss
Task 6	A parameter update

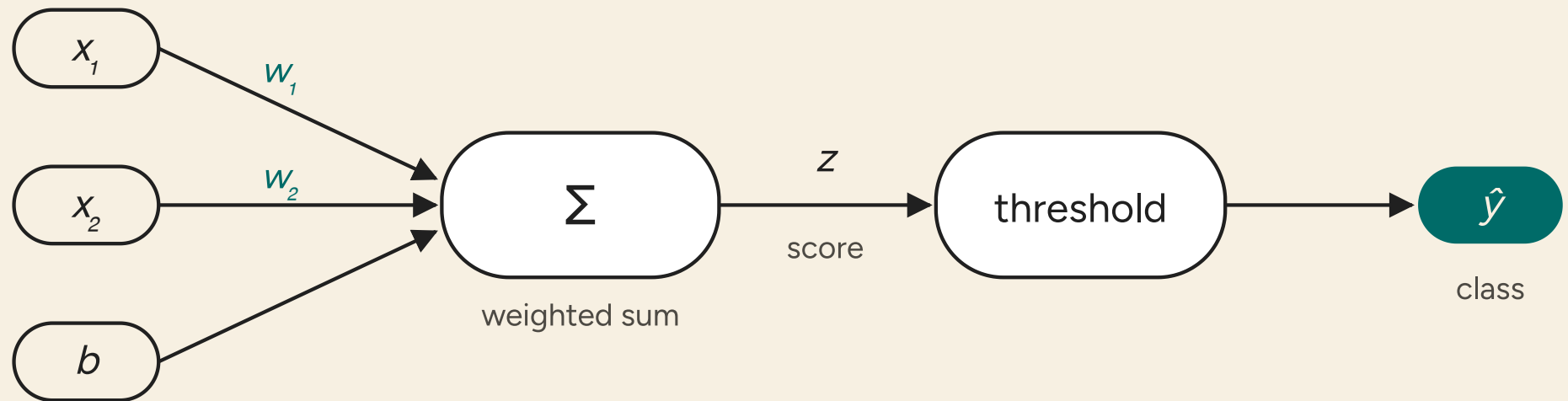
If a weight has a positive loss gradient, should it increase or decrease?

The perceptron decision

$$z = w_1x_1 + w_2x_2 + b \quad \hat{y} = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Weights control each input's contribution.

Bias shifts the decision threshold.



An AND classifier

$$w_1 = w_2 = 1$$

$$b = -1.5$$

AND outputs 1 only when both inputs are 1.

x_1	x_2	Score z	Prediction $\hat{y}$
0	0	-1.5	0
0	1	-0.5	0
1	0	-0.5	0
1	1	0.5	1

The decision boundary

$$w_1x_1 + w_2x_2 + b = 0$$

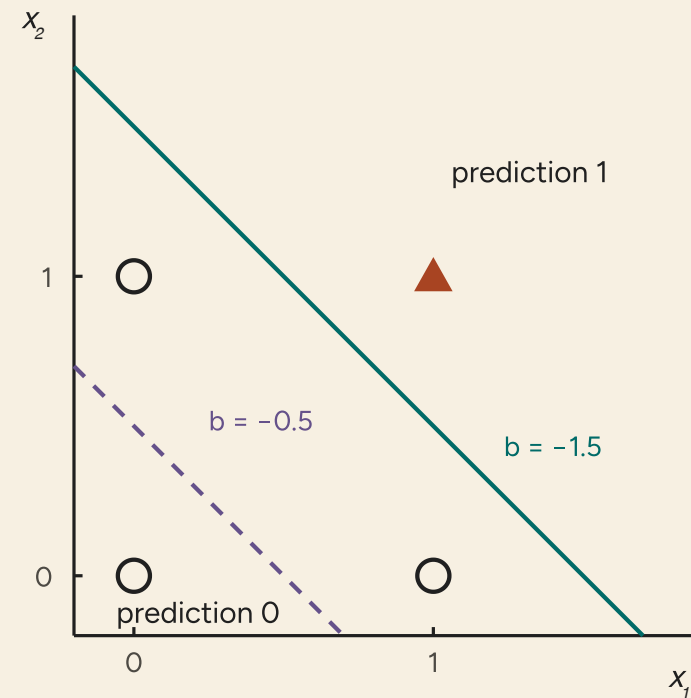
$$x_1 + x_2 = 1.5$$

Region labels refer to the solid line $b = -1.5$.
The dashed line $b = -0.5$ is a changed classifier with a larger positive region.

Increasing b expands the positive region in this example.

○ class 0 — hollow circle

▲ class 1 — filled triangle



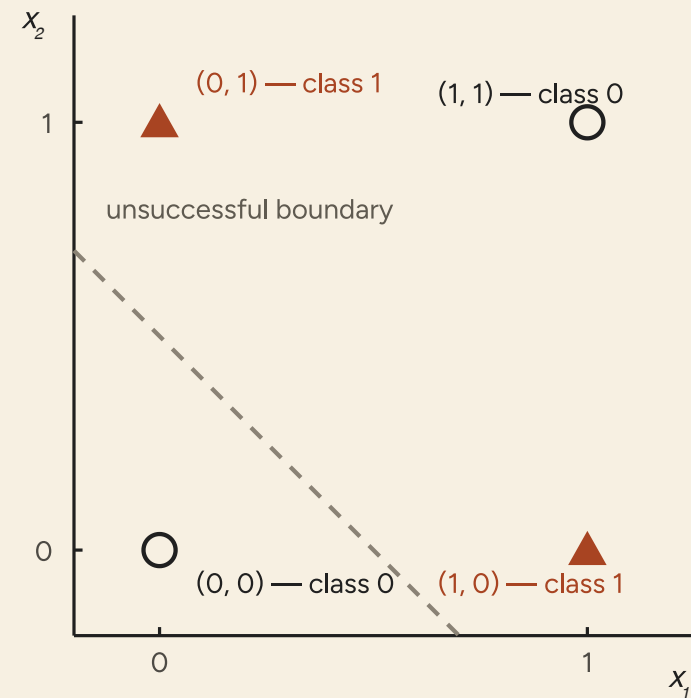
The XOR limitation

XOR outputs 1 when exactly one input is 1.

A single straight boundary cannot separate these classes.

○ class 0 — hollow circle

▲ class 1 — filled triangle



Learning from a mistake

CLASSICAL PERCEPTRON UPDATE

$$e = y - \hat{y}$$

$$\mathbf{w}_{\text{new}} = \mathbf{w} + \eta e \mathbf{x} \quad b_{\text{new}} = b + \eta e$$

Correct prediction: $e = 0$, so no update.

$\eta > 0$ controls the size of the correction.

One perceptron update

NEW STARTING PARAMETERS — RESET FROM THE AND SOLUTION

$\mathbf{w} = [0, 0], \quad b = 0, \quad \mathbf{x} = [1, 1], \quad y = 1, \quad \eta = 0.5$

QUANTITY	BEFORE	AFTER (UPDATED)
Weights	[0, 0]	[0.5, 0.5]
Bias	0	0.5
Score on this input	0	1.5
Prediction	0	1

$e = 1, \quad z_{\text{new}} = 0.5 + 0.5 + 0.5 = 1.5$

One example is corrected. This is not a trained AND classifier.

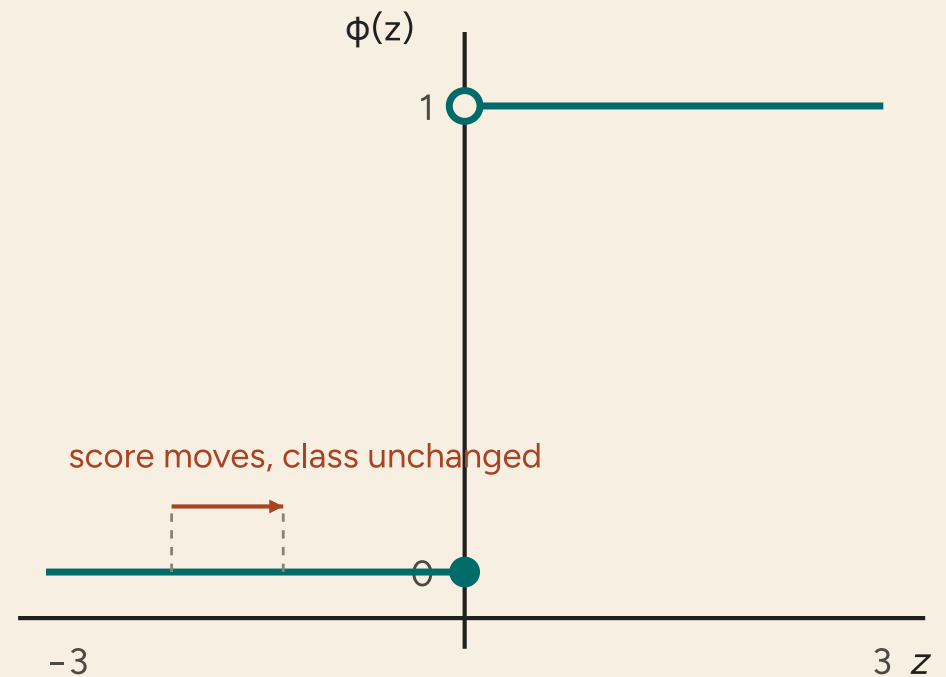
The hard threshold and its derivative

STEP ACTIVATION

$$\phi(z) = \mathbf{1} [z > 0]$$

Derivative: 0 away from zero, undefined at zero.

Small changes in the score usually leave the class unchanged.



A differentiable neuron

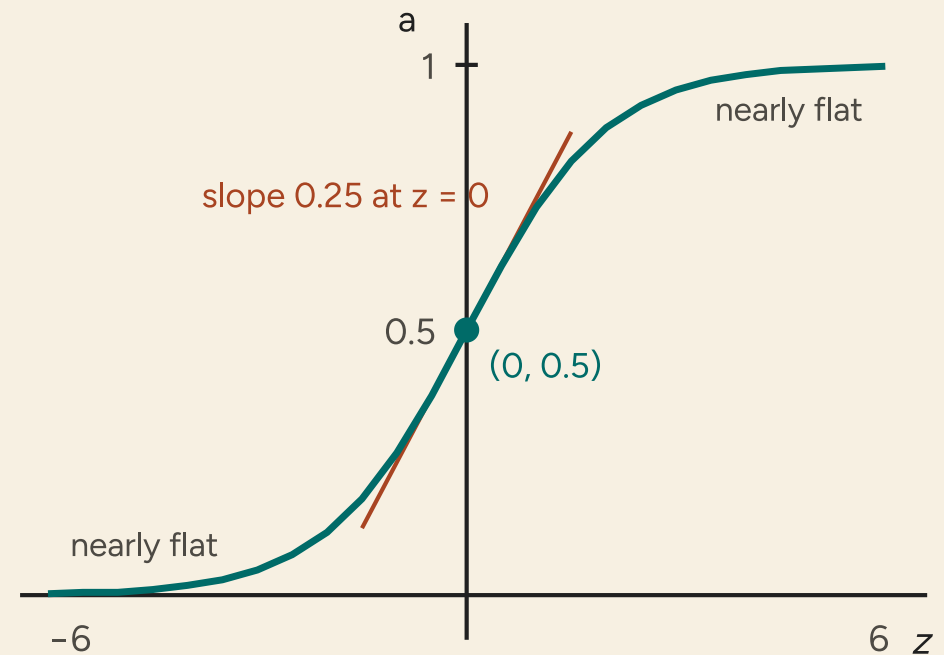
$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma'(z) = a(1 - a)$$

A small change in z changes the continuous prediction a .

The derivative becomes small in the saturated regions.

The continuous prediction a enters the loss.



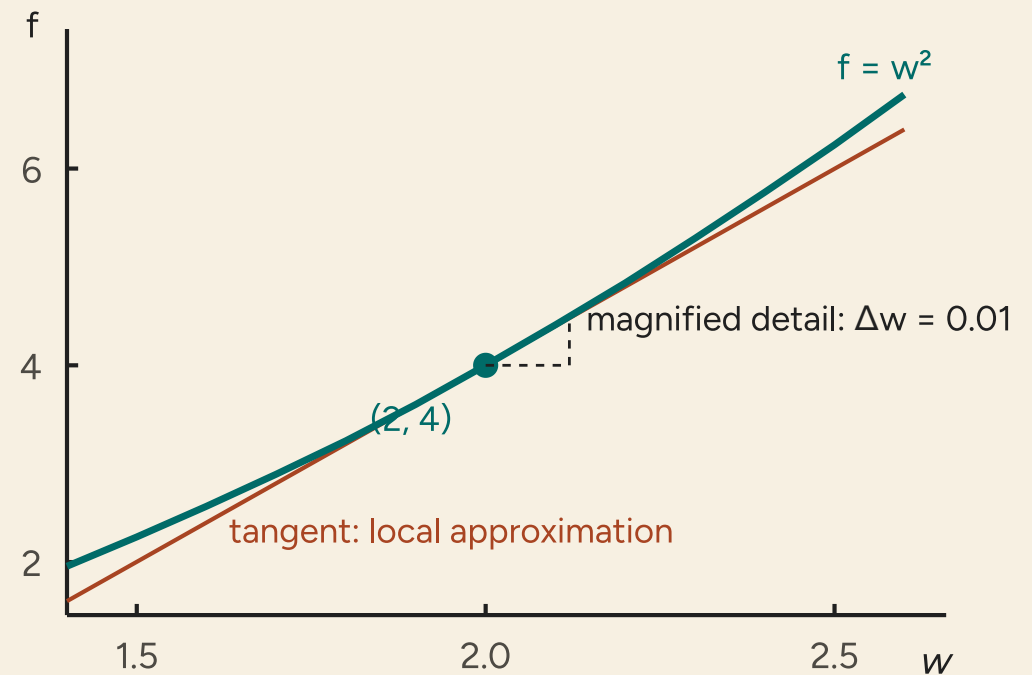
A derivative measures local sensitivity

$$f(w + \Delta w) \approx f(w) + f'(w) \Delta w$$

Example: $f(w) = w^2$, $w = 2$, $f'(w) = 4$.

Linear estimate = 0.04 for $\Delta w = 0.01$.

Exact change = 0.0401.



Partial derivatives and the gradient

$$f(w_1, w_2) = w_1^2 + 3w_1w_2 + 2w_2^2$$

$$\nabla f = \begin{bmatrix} 2w_1 + 3w_2 \\ 3w_1 + 4w_2 \end{bmatrix} \begin{matrix} \leftarrow \partial f / \partial w_1 \\ \leftarrow \partial f / \partial w_2 \end{matrix} \quad \nabla f(1, 2) = \begin{bmatrix} 8 \\ 11 \end{bmatrix}$$

Each partial derivative changes one coordinate while holding the other fixed.

Each component measures sensitivity to one parameter.

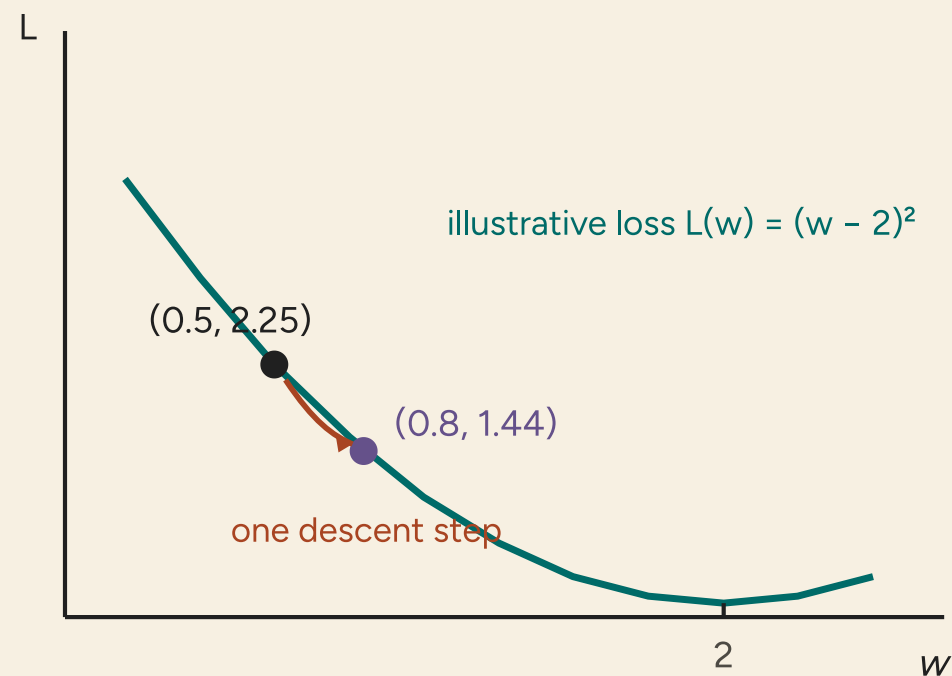
The direction of a parameter update

$$\theta_{\text{new}} = \theta - \eta \nabla L(\theta)$$

If $\partial L / \partial w = -3$ and $\eta = 0.1$, how does w change?

$$w_{\text{new}} = 0.5 - 0.1(-3) = 0.8$$

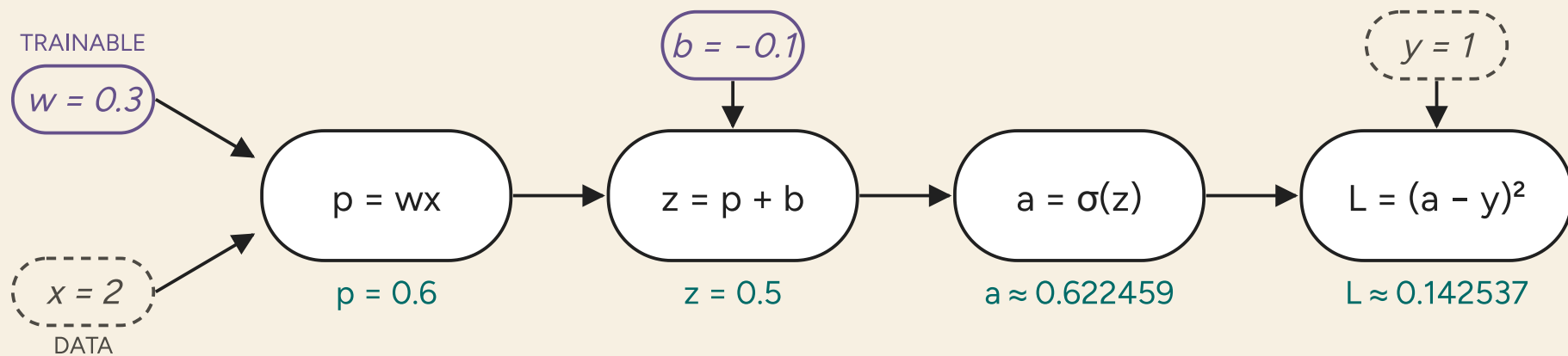
A sufficiently small step follows local descent. An excessive step can increase loss.



The homework neuron

$$z = wx + b \quad a = \sigma(z) \quad L = (a - y)^2$$

$$x = 2, \quad y = 1, \quad w = 0.3, \quad b = -0.1$$



forward values

How does a change in w reach the loss?

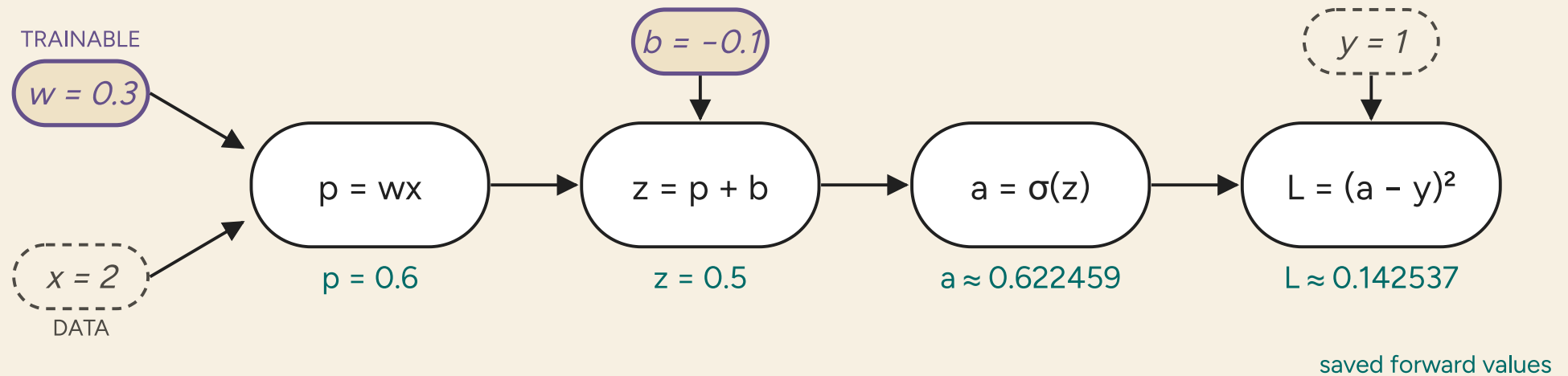
Lecture 1 checkpoint

- 1 How do z and $\hat{y}$ differ in a perceptron?
- 2 Why is the hard threshold difficult for gradient-based training?
- 3 Why does gradient descent subtract the gradient?

Forward values for the backward pass

LECTURE 2 BEGINS

$x = 2$, $y = 1$, $w = 0.3$, $b = -0.1$



Goal: calculate $\partial L / \partial w$ and $\partial L / \partial b$.

Local derivatives

OPERATION	LOCAL DERIVATIVES
$z = p + b$	$\partial z / \partial p = 1, \quad \partial z / \partial b = 1$
$p = wx$	$\partial p / \partial w = x, \quad \partial p / \partial x = w$
$L = (a - y)^2$	$\partial L / \partial a = 2(a - y)$
$a = \sigma(z)$	$\partial a / \partial z = a(1 - a)$

A local derivative describes one operation.

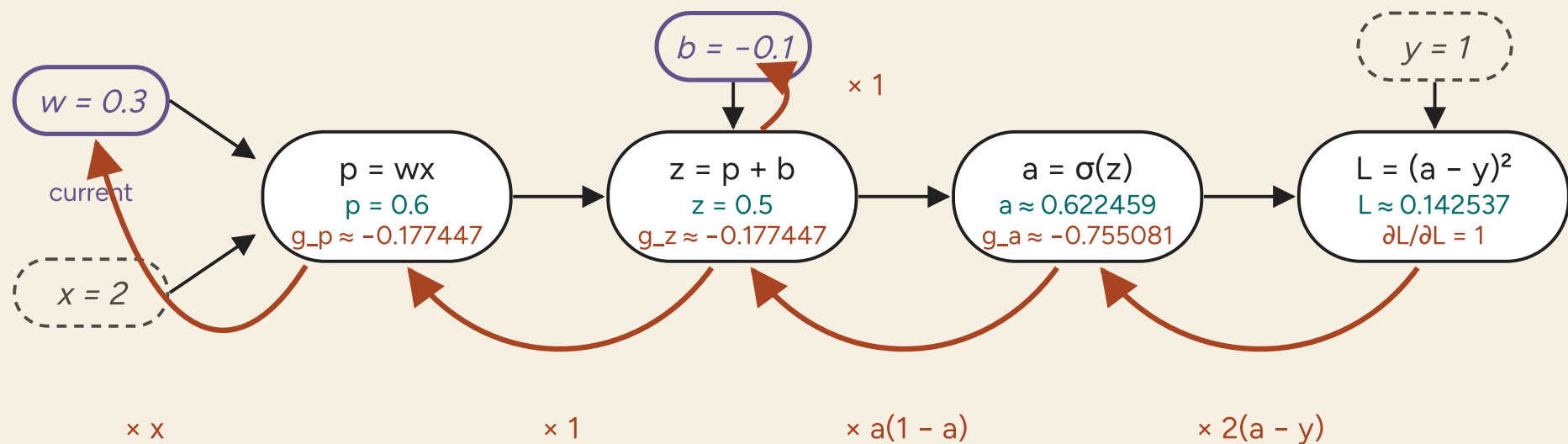
$a(1 - a)$ is a local factor, not the loss gradient $\partial L / \partial z$.

The chain rule through a neuron

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial p} \cdot \frac{\partial p}{\partial w}$$

≈ -0.354894 $2(a - y)$ $a(1 - a)$ 1 x

Gradient to an input = incoming loss gradient $\times$ local derivative.



Forward value

Node label: loss gradient · Connection label: $\times$ local factor

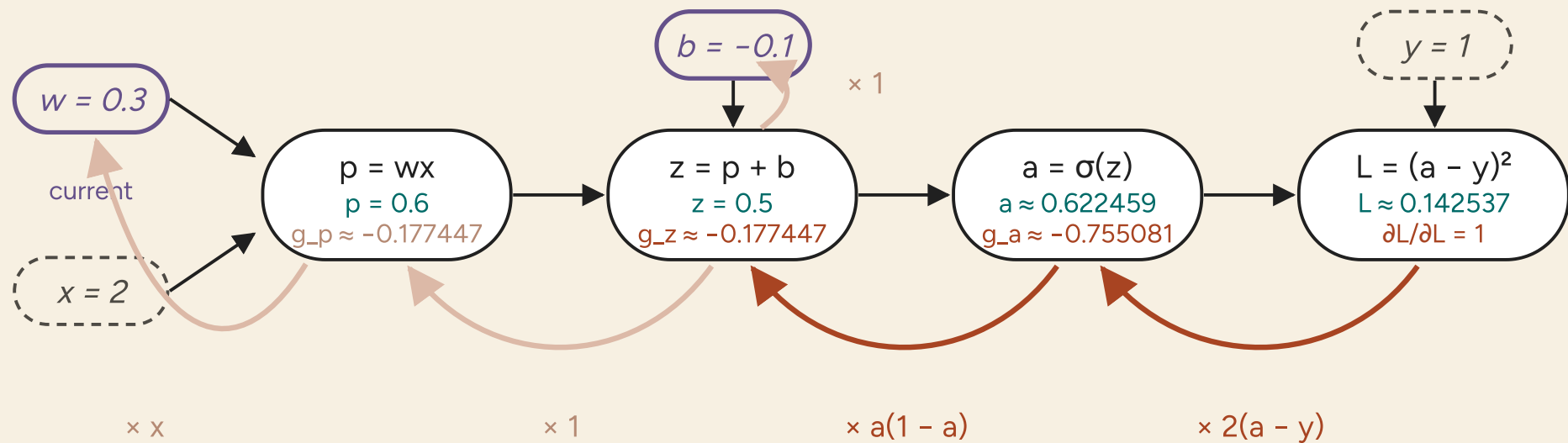
Trainable parameter

The loss sensitivity reaches z

$$\frac{\partial L}{\partial a} = 2(a - y) \approx -0.755081 \quad \frac{\partial a}{\partial z} = a(1 - a) \approx 0.235004$$

$$g_z \equiv \frac{\partial L}{\partial z} = 2(a - y) \cdot a(1 - a) \approx -0.177447$$

Increasing z slightly lowers this example's loss.



Forward value

Node label: loss gradient · Connection label: $\times$ local factor

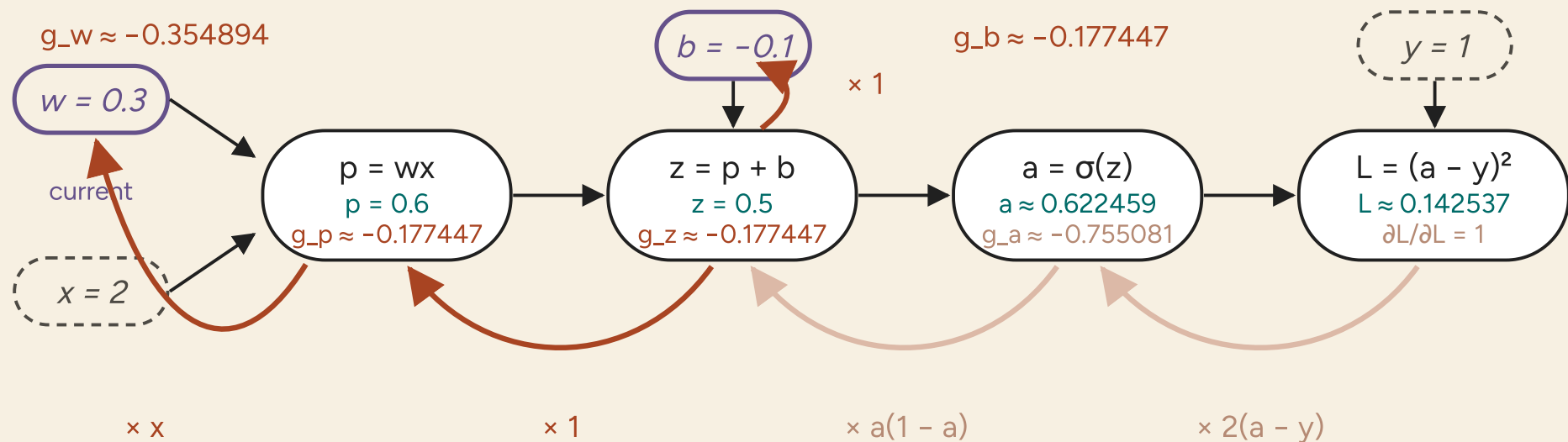
Trainable parameter

Weight and bias gradients

$$\partial z / \partial p = 1, \quad \partial p / \partial w = x = 2, \quad \partial z / \partial b = 1$$

$$\frac{\partial L}{\partial w} = g_z \cdot x \approx -0.354894 \qquad \frac{\partial L}{\partial b} = g_z \cdot 1 \approx -0.177447$$

Why is the weight gradient twice the bias gradient?



One gradient-descent update

$$\eta = 0.1 \quad w_{\text{new}} = w - \eta \partial L / \partial w \quad b_{\text{new}} = b - \eta \partial L / \partial b$$

QUANTITY	BEFORE (CURRENT)	AFTER (UPDATED)
w	0.3	≈ 0.335489
b	-0.1	≈ -0.082255
L	≈ 0.142537	≈ 0.127397

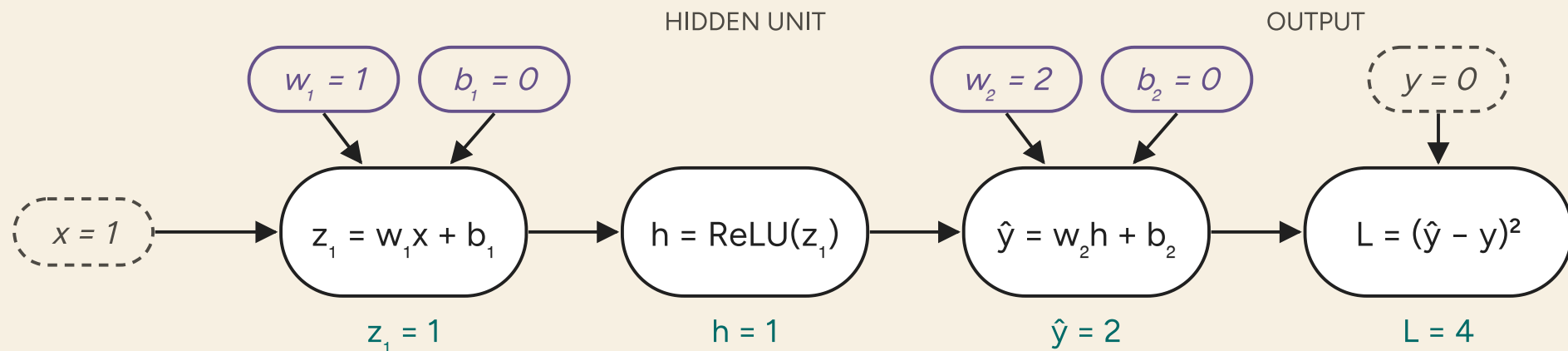
Compute all gradients, then update the parameters.

A network with one hidden unit

$$z_1 = w_1x + b_1 \quad h = \text{ReLU}(z_1) \quad \hat{y} = w_2h + b_2 \quad L = (\hat{y} - y)^2$$

New example: one hidden ReLU unit with a linear output. Linear output: $\hat{y}$ can exceed 1.

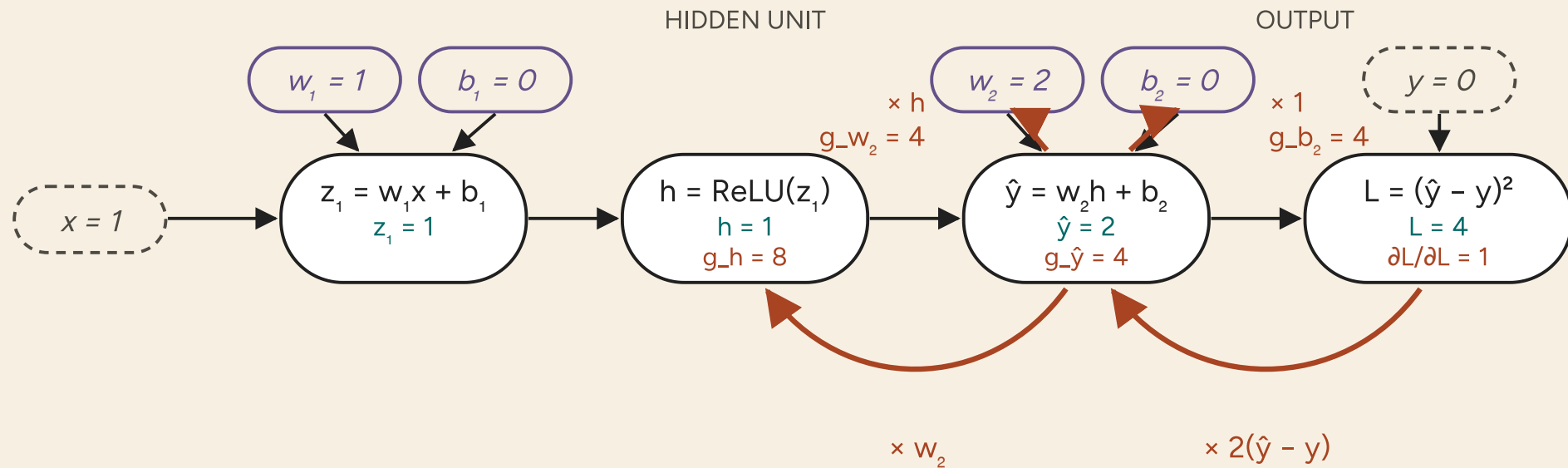
$$x = 1, y = 0, w_1 = 1, b_1 = 0, w_2 = 2, b_2 = 0$$



Backward through the output

$$\partial L / \partial \hat{y} = 2(\hat{y} - y) = 4 \quad \partial L / \partial w_2 = 4 \cdot h = 4 \quad \partial L / \partial b_2 = 4 \cdot 1 = 4 \quad \partial L / \partial h = 4 \cdot w_2 = 8$$

The weight gradient uses h . The gradient passed to h uses w_2 .



Forward value

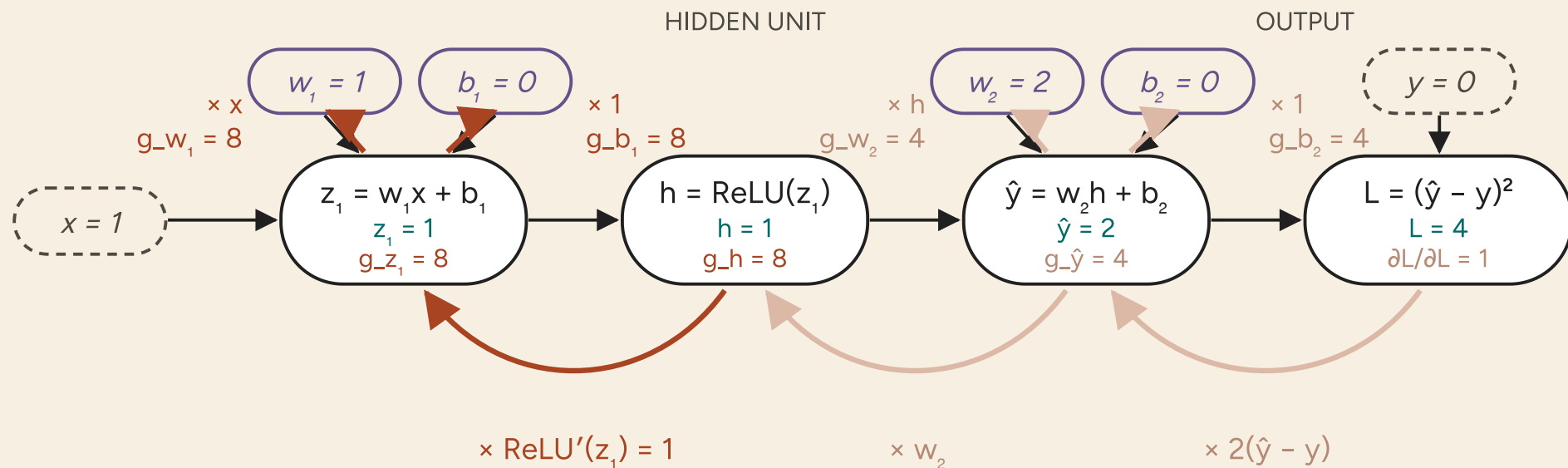
Node label: loss gradient · Connection label: $\times$ local factor

Trainable parameter

Backward through the hidden unit

$$z_1 = 1 > 0, \text{ReLU}'(z_1) = 1 \quad \partial L / \partial z_1 = 8 \cdot 1 = 8 \quad \partial L / \partial w_1 = 8x = 8 \quad \partial L / \partial b_1 = 8$$

$$\partial L / \partial w_1 = \underset{4}{2(\hat{y} - y)} \cdot \underset{2}{w_2} \cdot \underset{1}{\text{ReLU}'(z_1)} \cdot \underset{1}{x} = 8$$



Forward value

Node label: loss gradient · Connection label: $\times$ local factor

Trainable parameter

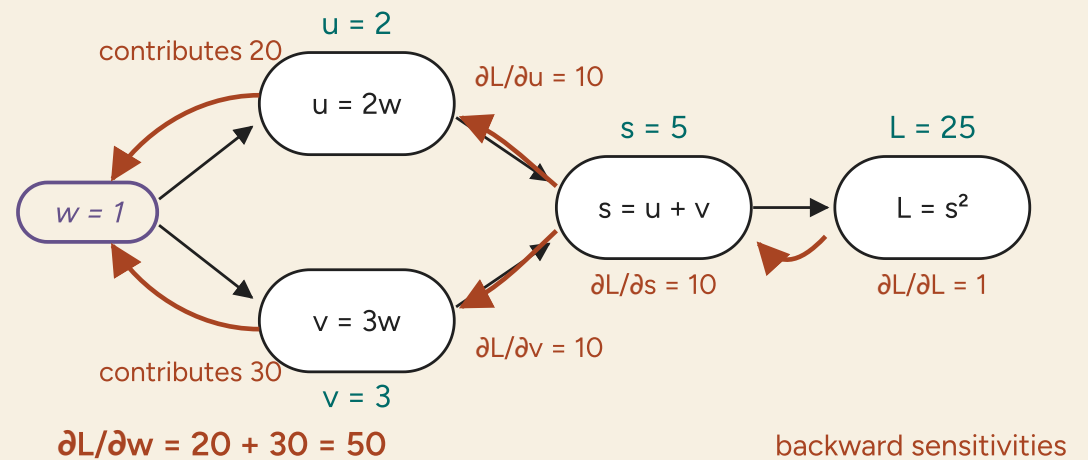
Gradients add across paths

$$u = 2w, \quad v = 3w, \quad s = u + v$$

$$L = s^2, \quad w = 1$$

$$\partial L / \partial w = 10 \times 2 + 10 \times 3 = 50$$

Multiply along each path. Add contributions from different paths.



The backpropagation procedure

- 1 Run forward and keep needed values.
- 2 Start backward with $\partial L / \partial L = 1$.
- 3 Visit operations in reverse dependency order.
- 4 Multiply by local derivatives and add contributions.

Backpropagation applies the chain rule in reverse order and reuses intermediate gradients.

$$\partial L / \partial w = 20 + 30 = 50$$

Numerical gradient checking

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon, b) - L(w - \varepsilon, b)}{2\varepsilon}$$

METHOD

ROLE

Finite differences

Estimate a derivative by perturbing a parameter

Backpropagation

Compute derivatives using the chain rule

$\varepsilon = 10^{-5}$ Hold b fixed while checking w .

For the homework neuron, both give approximately -0.354894 .

The training loop

CONCEPTUAL PSEUDOCODE

```
prediction = forward(data, parameters)
```

```
loss = objective(prediction, target)
```

```
gradients = backward(loss)
```

```
parameters = parameters - learning_rate * gradients
```

Backward computes gradients. The optimizer updates parameters.

A batch of examples

$$L_{\text{batch}}(\theta) = \frac{1}{N} \sum_{i=1}^N L_i(\theta) \quad \nabla_{\theta} L_{\text{batch}} = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L_i$$

ELEMENTWISE SQUARED-ERROR EXAMPLE

```
assert prediction.shape == target.shape
```

A residual needs one matching prediction and target per example.

Backpropagation checkpoint

For $a = \sigma(wx + b)$ and $L = (a - y)^2$:

PROPOSED GRADIENT — INTENTIONALLY INCOMPLETE

$$\partial L / \partial w = 2(a - y)x$$

AFTER DISCUSSION

$$\partial L / \partial w = 2(a - y) a(1 - a) x$$

- 1 Which factor is missing?
- 2 What does backward compute?
- 3 What happens when two paths reach the same parameter?

NEXT WEEK

Week 3: Multi-Layer Perceptrons + Loss Functions

More neurons use the same chain rule.

The chosen loss determines the initial backward signal.

Sigmoid derivative derivation

$$a = \sigma(z) = (1 + e^{-z})^{-1}$$

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

$$\sigma'(z) = \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}} \right) = a(1 - a)$$

Hidden-unit update reference

$\eta = 0.01$

PARAMETER	BEFORE (CURRENT)	GRADIENT	AFTER (UPDATED)
w_1	1	8	0.92
b_1	0	8	-0.08
w_2	2	4	1.96
b_2	0	4	-0.04

$\hat{y}_{\text{new}} = 1.6064, \quad L_{\text{new}} = 2.58052096 < 4$

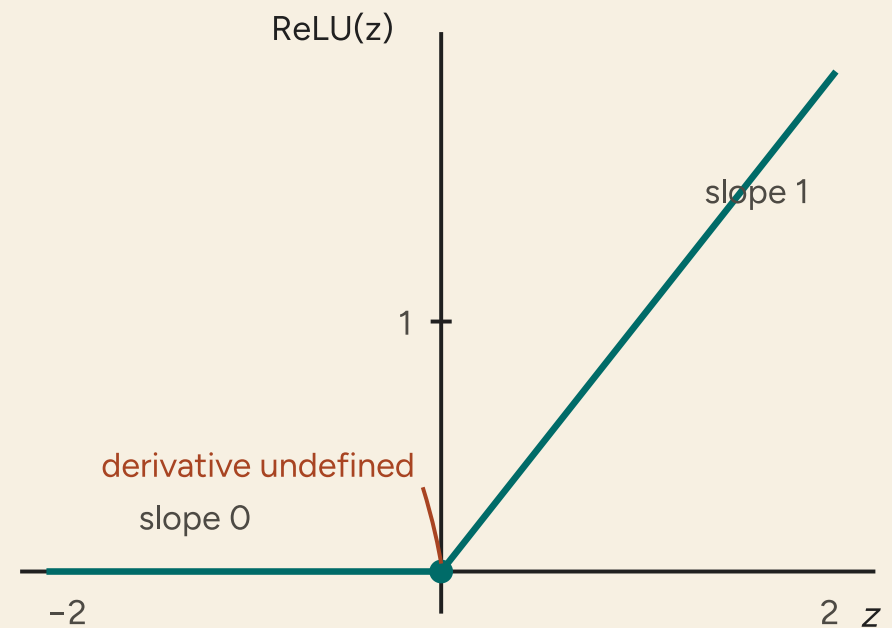
ReLU and zero gradients

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}'(z) = \begin{cases} 0, & z < 0 \\ 1, & z > 0 \end{cases}$$

At $z = 0$, ReLU is not differentiable.

For $z < 0$, this unit passes zero sensitivity backward to z .



Numerical reference values

Homework neuron: $x = 2$, $y = 1$, $w = 0.3$, $b = -0.1$

QUANTITY	VALUE
z	0.5
a	0.6224593312
L	0.1425369566
$\partial L / \partial w$	-0.3548938347
$\partial L / \partial b$	-0.1774469173
L after one update, $\eta = 0.1$	0.1273974585